

```

import anthropic
import requests
import json
import os
import random
from datetime import datetime
from urllib.parse import quote

# =====
# КОНФИГ — заполни переменные окружения в GitHub Secrets
# =====
ANTHROPIC_KEY      = os.environ["ANTHROPIC_API_KEY"]
TELEGRAM_TOKEN     = os.environ["TELEGRAM_BOT_TOKEN"]
TELEGRAM_CHANNEL   = os.environ["TELEGRAM_CHANNEL_ID"]    # например: @your_channel
PINTEREST_TOKEN    = os.environ["PINTEREST_ACCESS_TOKEN"]
PINTEREST_BOARD    = os.environ["PINTEREST_BOARD_ID"]

client = anthropic.Anthropic(api_key=ANTHROPIC_KEY)

# =====
# ТЕМЫ — вокруг когнитивной усталости, не раскрывают гайд
# =====
TOPICS = [
    "Почему мозг устаёт не от работы, а от неопределённости",
    "5 сигналов, что твой фокус на нуле — и как это исправить за день",
    "Цифровой туман: что происходит с мозгом после 14:00",
    "Почему усилия воли не работают против выгорания",
    "Дофаминовая ловушка: как соцсети крадут твою концентрацию",
    "Скрытые причины 'тумана в голове' после продуктивного утра",
    "Что наука говорит о когнитивной перегрузке в 2026 году",
    "Почему ты читаешь 2 часа и не помнишь ничего",
    "Холодный душ, свет и дыхание: нейробиология утреннего старта",
    "Как восстановить концентрацию без цифрового детокса",
    "Executive function: почему принимать решения к вечеру так тяжело",
    "Рабочая память перегружена — признаки и быстрые решения",
    "Глимфатическая система: мозг очищается только ночью",
    "Почему 4 часа фокуса лучше 10 часов 'присутствия'",
    "Как остановить бесконечный скролл на нейробиологическом уровне",
    "Руминация: почему мысли крутятся по кругу и как их остановить",
    "Магний, L-теанин, омега-3 — что реально работает для мозга",
    "Как вибрация телефона снижает IQ на 10 пунктов",
    "Переключение задач: невидимая цена для продуктивности",
    "Почему выгорание — это не лень, а нейробиологический сбой",
]

```

```

# =====
# 1. ГЕНЕРАЦИЯ КОНТЕНТА
# =====
def generate_content(topic: str) -> dict:
    prompt = f"""Ты эксперт по нейронауке и продуктивности. Создай контент на тему

    ВАЖНО: не давай полных инструкций и протоколов — только инсайты, факты, интригу.
    Цель — вызвать интерес к теме, а не решить проблему полностью.

    Верни ТОЛЬКО валидный JSON без markdown и без пояснений:
    {{
        "telegram_post": "пост 150-220 слов с эмодзи, живой язык, 1 конкретный факт/инс
        "pinterest_title": "заголовок до 90 символов, цепляющий",
        "pinterest_description": "описание 200-300 символов, польза + интрига, без спой
        "image_prompt": "prompt in English for image generation, minimalist style, brai
    }}"""

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=1000,
        messages=[{"role": "user", "content": prompt}]
    )

    raw = response.content[0].text.strip()
    raw = raw.replace("`json", "").replace("`", "").strip()
    return json.loads(raw)

# =====
# 2. ГЕНЕРАЦИЯ КАРТИНКИ (Pollinations — бесплатно, без ключа)
# =====
def generate_image(prompt: str) -> bytes:
    enhanced = f"{prompt}, high quality, 4k, editorial photography style, clean b
    url = f"https://image.pollinations.ai/prompt/{quote(enhanced)}?width=1024&hei
    response = requests.get(url, timeout=60)
    response.raise_for_status()
    return response.content

# =====
# 3. ПУБЛИКАЦИЯ В TELEGRAM
# =====
def post_to_telegram(text: str, image_bytes: bytes) -> bool:
    url = f"https://api.telegram.org/bot{TELEGRAM_TOKEN}/sendPhoto"
    resp = requests.post(
        url,
    
```

```

        data={"chat_id": TELEGRAM_CHANNEL, "caption": text, "parse_mode": "HTML"}
        files={"photo": ("image.jpg", image_bytes, "image/jpeg")},
        timeout=30
    )
    result = resp.json()
    if not result.get("ok"):
        print(f"❌ Telegram error: {result}")
        return False
    print(f"✅ Telegram: опубликовано")
    return True

# =====
# 4. ПУБЛИКАЦИЯ В PINTEREST
# =====
def post_to_pinterest(title: str, description: str, image_bytes: bytes) -> bool:
    # Шаг 1: получить upload URL
    media_resp = requests.post(
        "https://api.pinterest.com/v5/media",
        headers={
            "Authorization": f"Bearer {PINTEREST_TOKEN}",
            "Content-Type": "application/json"
        },
        json={"media_type": "image_base64"},
        timeout=30
    )
    if media_resp.status_code != 201:
        print(f"❌ Pinterest media init error: {media_resp.text}")
        return False

    media_data = media_resp.json()
    upload_url = media_data["upload_url"]
    upload_params = media_data.get("upload_parameters", {})
    media_id = media_data["media_id"]

    # Шаг 2: загрузить картинку
    files = {k: (None, v) for k, v in upload_params.items()}
    files["file"] = ("image.jpg", image_bytes, "image/jpeg")
    upload_resp = requests.post(upload_url, files=files, timeout=60)
    if upload_resp.status_code not in (200, 204):
        print(f"❌ Pinterest upload error: {upload_resp.text}")
        return False

    # Шаг 3: создать пин
    pin_resp = requests.post(
        "https://api.pinterest.com/v5/pins",
        headers={

```

```

        "Authorization": f"Bearer {PINTEREST_TOKEN}",
        "Content-Type": "application/json"
    },
    json={
        "board_id": PINTEREST_BOARD,
        "title": title,
        "description": description,
        "media_source": {"source_type": "image_id", "media_id": media_id}
    },
    timeout=30
)
if pin_resp.status_code != 201:
    print(f"❌ Pinterest pin error: {pin_resp.text}")
    return False

print(f"✅ Pinterest: опубликовано")
return True

# =====
# 5. ГЛАВНЫЙ ЗАПУСК
# =====
def run():
    now = datetime.utcnow().strftime("%Y-%m-%d %H:%M UTC")
    topic = random.choice(TOPICS)
    print(f"\n🤖 [{now}] Тема: {topic}")

    try:
        print("✍️ Генерирую контент...")
        content = generate_content(topic)

        print("🎨 Генерирую картинку...")
        image = generate_image(content["image_prompt"])

        print("📤 Публикую...")
        post_to_telegram(content["telegram_post"], image)
        post_to_pinterest(content["pinterest_title"], content["pinterest_descript

        print("🎉 Готово!\n")

    except Exception as e:
        print(f"❌ Критическая ошибка: {e}")
        raise

if __name__ == "__main__":
    run()

```

